

Cowrie Honeypot and Log Analysis

Cybersecurity Internship Project Report

Platform	Honeypot	Analysis	Lab Scope
Kali Linux VM	Cowrie	Python + JSON	127.0.0.1 localhost

Abstract

This project deployed a Cowrie SSH honeypot in a controlled Kali Linux lab to simulate a vulnerable Linux service, capture connection and login activity, record commands entered inside the emulated shell, and analyze the resulting JSON logs with Python. The project combined honeypot deployment with SOC-style log analysis by summarizing connections, usernames, passwords, source IPs, and observed commands. All traffic was generated locally from 127.0.0.1 for safe testing; the honeypot was not exposed to the public internet.

Introduction

Honeypots are defensive security systems designed to attract and observe suspicious activity without exposing a real production host. Cowrie provides an emulated SSH/Telnet environment that can record credentials, commands, session behavior, and structured event data. For this project, Cowrie was used as the monitoring component and a custom Python script was added to convert raw JSON telemetry into an easy-to-read activity summary.

Tools Used

- Kali Linux running in Oracle VirtualBox
- Cowrie SSH/Telnet honeypot
- Python 3 and a Python virtual environment (venv)
- OpenSSH client for generating controlled test sessions
- Cowrie text logs and JSON event logs

Steps Involved in Building the Project

- **1. Environment preparation:** Created a dedicated non-root Cowrie user and a separate Python virtual environment.
- **2. Installation:** Installed Cowrie with pip, initialized its configuration, and started the honeypot service.
- **3. Validation:** Confirmed Cowrie was running and listening for SSH connections on TCP port 2222.
- **4. Controlled testing:** Generated localhost SSH sessions using test, root, and admin accounts with fake lab passwords.
- **5. Session activity:** Executed commands such as whoami, ls, uname -a, pwd, and cat /etc/passwd inside Cowrie's emulated shell.
- **6. Logging:** Reviewed cowrie.log and cowrie.json to confirm that login attempts, source IPs, credentials, commands, and session events were recorded.
- **7. Analysis:** Built analyze_logs.py to parse Cowrie JSON events and summarize security telemetry.

```
cowrie-env (cowrie@kali) [~/honeypot]
$ tail -30 var/log/cowrie/cowrie.log
2026-09-27T01:54:09.500268Z [HoneyPotSSHTransport,1,127.0.0.1] Command found: exit
2026-09-27T01:54:09.500718Z [HoneyPotSSHTransport,1,127.0.0.1] exitCode: 0
2026-09-27T01:54:09.501201Z [ssh,face0e603382b,127.0.0.1] Closing TTY Log: var/lib/cowrie/tty/2638f1c2018567a46a4cae049dd90db2d468e1538d60d328f2707d071f73c5 after 2377 milliseconds
2026-09-27T01:54:09.501310Z [HoneyPotSSHTransport,1,127.0.0.1] sending close 0
2026-09-27T01:54:09.503173Z [HoneyPotSSHTransport,1,127.0.0.1] remote close
2026-09-27T01:54:09.503401Z [HoneyPotSSHTransport,1,127.0.0.1] Got remote error, code 11 reason: b'disconnected by user'
2026-09-27T01:54:09.503752Z [HoneyPotSSHTransport,1,127.0.0.1] avatar root logging out
2026-09-27T01:54:09.503862Z [HoneyPotSSHTransport,1,127.0.0.1] connection lost
2026-09-27T01:54:09.504030Z [ssh,face0e603382b,127.0.0.1] Connection lost after 7162 milliseconds
2026-09-27T01:54:18.431760Z [ssh,b0188303f39b,127.0.0.1] New connection: 127.0.0.1:38210 (127.0.0.1:2222) [session: b0188303f39b]
2026-09-27T01:54:18.432030Z [ssh,b0188303f39b,127.0.0.1] Remote SSH version: SSH-2.0-OpenSSH_10.3p1 Debian-5
2026-09-27T01:54:18.433893Z [ssh,b0188303f39b,127.0.0.1] SSH client hash fingerprint: eeca2460550b9ded084ecf2f70a75356
2026-09-27T01:54:21.149500Z [ssh,b0188303f39b,127.0.0.1] login attempt [admin/admin123] succeeded
2026-09-27T01:54:21.149756Z [HoneyPotSSHTransport,2,127.0.0.1] Initialized emulated server as architecture: linux-x64-lsb
2026-09-27T01:54:21.152761Z [HoneyPotSSHTransport,2,127.0.0.1] channel open
2026-09-27T01:54:21.248734Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransport,2,127.0.0.1] Handling pty request: b'xterm-256color' (28, 77, 0, 0)
2026-09-27T01:54:21.249009Z [ssh,b0188303f39b,127.0.0.1] Terminal Size: 77 28
2026-09-27T01:54:21.249881Z [ssh,b0188303f39b,127.0.0.1] request_env: COLORTERM=truecolor
2026-09-27T01:54:21.250039Z [ssh,b0188303f39b,127.0.0.1] request_env: LANG=en_US.UTF-8
2026-09-27T01:54:21.250161Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransport,2,127.0.0.1] Getting shell
2026-09-27T01:54:27.062980Z [ssh,b0188303f39b,127.0.0.1] CMD: exit
2026-09-27T01:54:27.064765Z [HoneyPotSSHTransport,2,127.0.0.1] Command found: exit
2026-09-27T01:54:27.064947Z [HoneyPotSSHTransport,2,127.0.0.1] exitCode: 0
2026-09-27T01:54:27.065250Z [ssh,b0188303f39b,127.0.0.1] Closing TTY Log: var/lib/cowrie/tty/2638f1c2018567a46a4cae049dd90db2d468e1538d60d328f2707d071f73c5 after 5815 milliseconds
2026-09-27T01:54:27.065335Z [HoneyPotSSHTransport,2,127.0.0.1] sending close 0
2026-09-27T01:54:27.066320Z [HoneyPotSSHTransport,2,127.0.0.1] remote close
2026-09-27T01:54:27.066452Z [HoneyPotSSHTransport,2,127.0.0.1] Got remote error, code 11 reason: b'disconnected by user'
2026-09-27T01:54:27.066686Z [HoneyPotSSHTransport,2,127.0.0.1] avatar admin logging out
2026-09-27T01:54:27.066756Z [HoneyPotSSHTransport,2,127.0.0.1] connection lost
2026-09-27T01:54:27.066855Z [ssh,b0188303f39b,127.0.0.1] Connection lost after 8635 milliseconds
```

Figure 1. Cowrie human-readable logs capturing SSH login and session activity.

Results and Observations

Metric	Observed Result
Total connections	4
Successful logins	4
Failed logins	0
Source IP	127.0.0.1 (localhost) - 4 connections
Username attempted	admin: 2, test: 1, root: 1
Passwords attempted	admin123: 2, password123: 1, hello123: 1
Commands observed	exit: 4, whoami: 1, ls: 1, uname -a: 1, pwd: 1, cat /etc/passwd: 1

Python Log Analyzer

The Python analyzer reads Cowrie's structured JSON log one event at a time. It uses each event's eventid to identify connection events, successful or failed logins, and command input. The script uses Python's Counter class to group repeated source IPs, usernames, passwords, and commands. The resulting summary was also saved to honeypot_analysis.txt for documentation.

```
(cowrie-env)(cowrie@kali)~[/honeypot]
$ python analyze_logs.py

=====
COWRIE HONEYPOT LOG ANALYSIS
=====

Total Connections: 4
Successful Logins: 4
Failed Logins: 0

Top Source IPs:
127.0.0.1: 4

Usernames Attempted:
admin: 2
test: 1
root: 1

Password Attempts:
admin123: 2
password123: 1
hello123: 1

Commands Observed:
exit: 4
whoami: 1
ls: 1
uname -a: 1
pwd: 1
cat /etc/passwd: 1

=====
Analysis Complete
=====
```

Figure 2. Python analyzer summarizing the locally generated Cowrie activity.

Conclusion

The project successfully deployed a Cowrie honeypot, generated controlled SSH activity, captured credentials and command behavior, and transformed the collected JSON logs into a readable security summary with Python. The work demonstrates practical blue-team skills including honeypot deployment, SSH monitoring, structured log review, security telemetry analysis, and basic Python automation. Because the activity was generated only from localhost, the results demonstrate functionality rather than real-world attacker trends. Future improvements could include repeated brute-force detection, visualization, threat-intelligence enrichment, alerting, and deployment in a properly isolated cloud lab.

Lab note: All usernames, passwords, commands, and source activity in this report were manually generated for testing. No real attacker credentials or public-internet traffic were collected.